

RELATÓRIO TÉCNICO

React

Características, vantagens, aplicações no mercado e exemplo de utilização em um projeto Web

Tecnologia estudada	React
Área	Desenvolvimento Front-end
Projeto prático	TaskFlow — aplicação de gerenciamento de tarefas
Ferramentas	Visual Studio Code, Vite, Git, GitHub e Vercel
Aluno	Vinícius Côrte

Relatório elaborado para apresentar os fundamentos do React, seus principais recursos, benefícios para o desenvolvimento de interfaces e sua presença em projetos Web modernos.

1. Introdução

O desenvolvimento Web moderno exige interfaces rápidas, organizadas e capazes de responder às ações do usuário sem a necessidade de recarregar continuamente a página. Nesse cenário, bibliotecas e frameworks de Front-end assumem um papel importante na construção de aplicações interativas. O React é uma das tecnologias mais utilizadas para esse objetivo e se destaca por permitir a criação de interfaces a partir de componentes reutilizáveis.

O React foi criado pelo Facebook, atualmente Meta, e tornou-se uma tecnologia de referência no ecossistema JavaScript. Embora tecnicamente seja apresentado como uma biblioteca para construção de interfaces de usuário, no mercado é frequentemente tratado como parte do conjunto de frameworks e ferramentas de Front-end. Ele pode ser utilizado sozinho ou combinado com outras tecnologias para formar aplicações completas.

Este relatório apresenta as principais características do React, suas vantagens, situações de uso, ferramentas relacionadas e exemplos de aplicação no mercado. Ao final, é apresentado um exemplo prático baseado no projeto TaskFlow, desenvolvido com React e Vite e preparado para publicação na plataforma Vercel.

2. O que é React?

React é uma biblioteca JavaScript voltada principalmente para a criação de interfaces de usuário. Sua principal ideia é dividir a interface em componentes independentes, permitindo que cada parte da aplicação tenha uma responsabilidade específica. Um botão, um formulário, um cabeçalho ou uma lista de tarefas, por exemplo, podem ser representados como componentes.

Essa abordagem favorece a reutilização de código e facilita a manutenção. Quando uma aplicação cresce, a separação por componentes ajuda a equipe a localizar problemas, alterar partes específicas e reaproveitar estruturas em diferentes telas.

2.1 Principais conceitos

- **Componentes:** partes reutilizáveis da interface que podem possuir estrutura, comportamento e estilo próprios.
- **JSX:** sintaxe que permite descrever a estrutura da interface dentro de arquivos JavaScript.
- **Props:** dados enviados de um componente para outro, normalmente do componente pai para o filho.
- **State:** informações que podem mudar durante a execução e provocar uma nova renderização da interface.
- **Hooks:** recursos como `useState` e `useEffect` que permitem trabalhar com estado e efeitos em componentes funcionais.

2.2 Funcionamento baseado em componentes

Em uma aplicação React, a interface pode ser organizada em uma árvore de componentes. Um componente principal pode reunir componentes menores, que por sua vez podem receber propriedades e controlar seus próprios estados. Essa estrutura torna o código mais previsível e facilita a evolução do projeto.

3. Principais características do React

O React possui características que ajudam a explicar sua popularidade no desenvolvimento Front-end. Entre elas estão a componentização, o gerenciamento de estado, a atualização eficiente da interface e a possibilidade de integração com diversas ferramentas do ecossistema JavaScript.

3.1 Componentização e reutilização

A componentização é uma das características centrais do React. Em vez de concentrar toda a interface em um único arquivo, o desenvolvedor pode criar componentes menores. Isso reduz a repetição de código e permite reutilizar elementos em diferentes páginas ou situações.

3.2 JSX

O JSX permite escrever uma estrutura semelhante ao HTML dentro do código JavaScript. Ele não é HTML puro: durante o processo de construção da aplicação, ferramentas como Vite transformam esse código para que o navegador consiga executá-lo. O JSX aproxima a descrição visual da lógica do componente, tornando o desenvolvimento mais direto.

```
function TaskItem({ task }) {
  return (
    <div className="task-item">
      <strong>{task.title}</strong>
      <span>{task.status}</span>
    </div>
  );
}
```

3.3 Estado e interatividade

O estado permite que a aplicação armazene informações que mudam com a interação do usuário. Um formulário pode guardar o texto digitado, uma lista pode registrar tarefas adicionadas e um botão pode alterar o estado de uma atividade. Quando o estado relevante muda, o React atualiza a interface correspondente.

3.4 Hooks

Hooks são funções disponibilizadas pelo React para trabalhar com recursos dos componentes funcionais. O `useState` é utilizado para armazenar estado local, enquanto o `useEffect` é usado em situações que envolvem efeitos, como determinadas operações após a renderização ou integração com recursos externos.

3.5 Ecossistema e ferramentas

React não precisa trabalhar sozinho. Projetos reais podem utilizar Vite para criação e build, bibliotecas de roteamento para navegação, soluções de estilização, ferramentas de testes, bibliotecas para formulários e serviços de backend ou APIs. Essa flexibilidade permite adaptar a arquitetura às necessidades do projeto.

4. Vantagens do uso do React

- **Reutilização:** componentes podem ser utilizados em várias partes da aplicação.
- **Manutenção:** a divisão em componentes facilita localizar e modificar funcionalidades.
- **Ecossistema amplo:** existem muitas ferramentas e bibliotecas complementares.
- **Produtividade:** a combinação de componentes, JSX e ferramentas modernas acelera o desenvolvimento.
- **Escalabilidade:** a arquitetura baseada em componentes pode ser aplicada desde pequenos projetos até aplicações maiores.
- **Mercado:** a tecnologia é amplamente conhecida por equipes de desenvolvimento Web, o que favorece a existência de materiais, cursos e oportunidades.

5. Aplicações do React no mercado

O React pode ser aplicado em diferentes tipos de sistemas Web. A escolha da tecnologia depende do problema, da arquitetura do projeto, da equipe e dos requisitos de desempenho e manutenção. Entre os cenários mais comuns estão sistemas administrativos, plataformas de conteúdo, lojas virtuais, dashboards e aplicações interativas.

Aplicação	Como o React pode ser utilizado
Dashboards	Construção de painéis com gráficos, filtros, indicadores e atualização dinâmica de dados.
E-commerce	Interfaces de produtos, carrinho, busca, filtros e etapas de compra.
Sistemas corporativos	Telas administrativas, cadastros, relatórios, permissões e formulários.
Redes e plataformas sociais	Feeds, comentários, notificações e componentes interativos.
Aplicações educacionais	Questionários, atividades, áreas do aluno, conteúdos e acompanhamento.

5.1 React em equipes e projetos profissionais

Em projetos profissionais, o React costuma ser combinado com práticas de controle de versão, revisão de código, testes automatizados, integração contínua e serviços de hospedagem. Ferramentas como Git e GitHub ajudam a controlar alterações, enquanto plataformas de deploy automatizam a publicação da aplicação.

É importante observar que React não define sozinho toda a arquitetura de uma aplicação. Em um projeto maior, a equipe precisa tomar decisões sobre roteamento, gerenciamento de estado, comunicação com APIs, autenticação, testes, organização de pastas, segurança e estratégia de publicação.

6. Limitações e cuidados

Apesar das vantagens, o uso do React também exige alguns cuidados. Uma aplicação pode ficar complexa se os componentes forem mal organizados, se houver estados desnecessários ou se o projeto acumular dependências sem planejamento. Também é importante conhecer JavaScript, HTML e CSS, pois o React é construído sobre esses fundamentos.

Outro ponto é que a flexibilidade do ecossistema pode aumentar a quantidade de decisões que o desenvolvedor precisa tomar. Por isso, antes de adicionar uma biblioteca, é recomendado avaliar se ela realmente resolve um problema do projeto.

7. React e Vite

Vite é uma ferramenta moderna de desenvolvimento e build que pode ser utilizada para criar aplicações React. Durante o desenvolvimento, ela oferece um servidor local rápido e, na produção,

gera arquivos otimizados para publicação. No projeto TaskFlow, essa combinação foi utilizada para estruturar a aplicação e preparar o build de produção.

8. Exemplo prático: projeto TaskFlow

Como exemplo de utilização do React em um projeto Web, foi desenvolvido o TaskFlow, uma aplicação de gerenciamento de tarefas. A proposta é apresentar uma interface em que o usuário possa visualizar, cadastrar e organizar tarefas de maneira simples.

O projeto foi criado no Visual Studio Code utilizando React com Vite. A estrutura foi organizada em componentes, permitindo separar responsabilidades e facilitar futuras alterações. Entre os componentes utilizados estão elementos para cabeçalho, formulário de tarefas, item individual e lista de tarefas.

8.1 Estrutura simplificada

```
taskflow-react-vercel/  
├── public/  
├── src/  
│   ├── components/  
│   │   ├── Header.jsx  
│   │   ├── TaskForm.jsx  
│   │   ├── TaskItem.jsx  
│   │   └── TaskList.jsx  
│   ├── App.jsx  
│   ├── App.css  
│   ├── index.css  
│   └── main.jsx  
├── index.html  
├── package.json  
└── vite.config.js
```

8.2 Exemplo de componente

Um componente de tarefa pode receber os dados por meio de props e apresentar as informações na tela. A separação desse elemento em um arquivo próprio facilita sua reutilização dentro da lista.

```
function TaskItem({ task }) {  
  return (  
    <article className="task-item">  
      <h3>{task.title}</h3>  
      <p>{task.description}</p>  
      <small>Status: {task.status}</small>  
    </article>  
  );  
}
```

8.3 Fluxo de desenvolvimento e publicação

- Criação do projeto React com Vite no Visual Studio Code.
- Desenvolvimento e organização dos componentes da interface.
- Teste local utilizando o servidor de desenvolvimento do Vite.
- Versionamento do código com Git.
- Publicação do repositório no GitHub.
- Configuração do projeto na Vercel.

- Execução do build de produção e disponibilização por uma URL pública.

8.4 Resultado

O exemplo demonstra como React pode ser utilizado para construir uma aplicação Web interativa e organizada. A combinação entre React, Vite, Git, GitHub e Vercel representa um fluxo de desenvolvimento próximo ao utilizado em muitos projetos modernos: desenvolver localmente, versionar o código, gerar uma versão de produção e disponibilizá-la online.

9. Conclusão

O React é uma tecnologia relevante para o desenvolvimento de interfaces Web modernas. Seu modelo baseado em componentes permite dividir aplicações em partes menores e reutilizáveis, enquanto recursos como JSX, estado e Hooks facilitam a criação de experiências interativas.

Entre seus principais benefícios estão a reutilização de componentes, a organização do código, a grande quantidade de ferramentas complementares e sua ampla utilização no mercado. Ao mesmo tempo, o desenvolvedor precisa manter uma boa organização do projeto e compreender os fundamentos de JavaScript, HTML e CSS.

O projeto TaskFlow demonstrou na prática como os conceitos do React podem ser aplicados. A utilização do Vite auxiliou na criação e no build do projeto, enquanto Git e GitHub permitiram o versionamento e a publicação do código. Por fim, a Vercel foi utilizada para disponibilizar a aplicação online, completando um fluxo de desenvolvimento e deploy.

Dessa forma, o estudo do React contribui não apenas para a criação de interfaces, mas também para a compreensão de práticas atuais de desenvolvimento Web. A tecnologia pode ser uma opção adequada para projetos de diferentes tamanhos, desde aplicações acadêmicas e protótipos até sistemas utilizados profissionalmente.

10. Referências e fontes para consulta

- React — documentação oficial: <https://react.dev/>
- Vite — documentação oficial: <https://vite.dev/>
- MDN Web Docs — JavaScript: <https://developer.mozilla.org/docs/Web/JavaScript>
- Git — documentação oficial: <https://git-scm.com/docs>
- Vercel — documentação oficial: <https://vercel.com/docs>

11. Considerações finais

A realização da atividade permitiu relacionar a pesquisa teórica sobre uma tecnologia Front-end com sua aplicação prática. A elaboração do TaskFlow mostrou que conhecer os conceitos de uma tecnologia é apenas uma parte do desenvolvimento: também são importantes o versionamento, a organização do projeto, o processo de build e a publicação da aplicação.

Como continuidade, o projeto pode evoluir com persistência de dados, integração com uma API, autenticação, filtros, categorias, testes automatizados e melhorias de acessibilidade e responsividade. Essas extensões permitiriam explorar outros recursos do ecossistema Web mantendo a base criada com React.